

现代循环神经网络

本章在上一章的基础上介绍两部分内容：第一部分是改进 RNN 本身，第二部分是將 RNN 用于序列生成任务中。

门控循环单元 (GRU)

之前的普通的 RNN 对于所有时间都一视同仁，**缺乏精细调控能力**，在下面几种情况中适用性较差：早期信息可能非常重要；有些输入可能不重要；序列中可能存在逻辑中断。

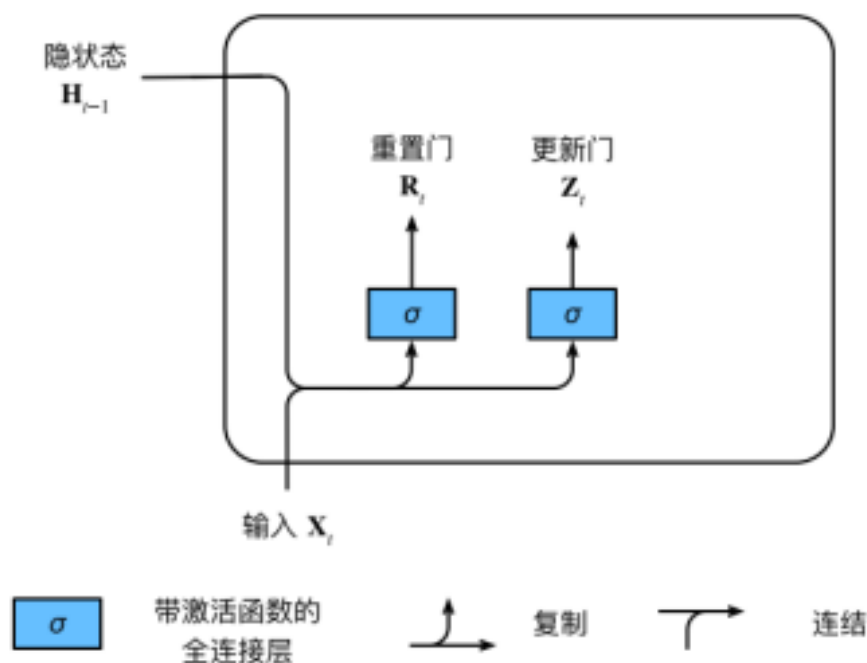
所以提出 GRU，目的：**让模型自己学习什么时候保留旧状态，什么时候接收新信息，什么时候忘掉过去。**

重要：具体构成：

Step1 重置门 R_t 和更新门 Z_t

首先，GRU 相比普通 RNN 不一样的地方在于，GRU 不再是直接将原状态和现输入综合为新状态，而是**先导出两个“门”（其实就是中间参数）**：重置门 R_t 和更新门 Z_t 。

（为什么是“门”？门有开有关，也可以半开半关，门开的大小决定了“通过”的多少：门开得越大，通过的越多，反之亦然）如下图所示



这两个门的意义是要看后面的 step2,3 来理解

计算公式为

$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r)$$

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z)$$

可以发现这两个门的计算公式是很相似的，而且门的取值都在 $(0, 1)$ 上（使用 sigmoid 函数）

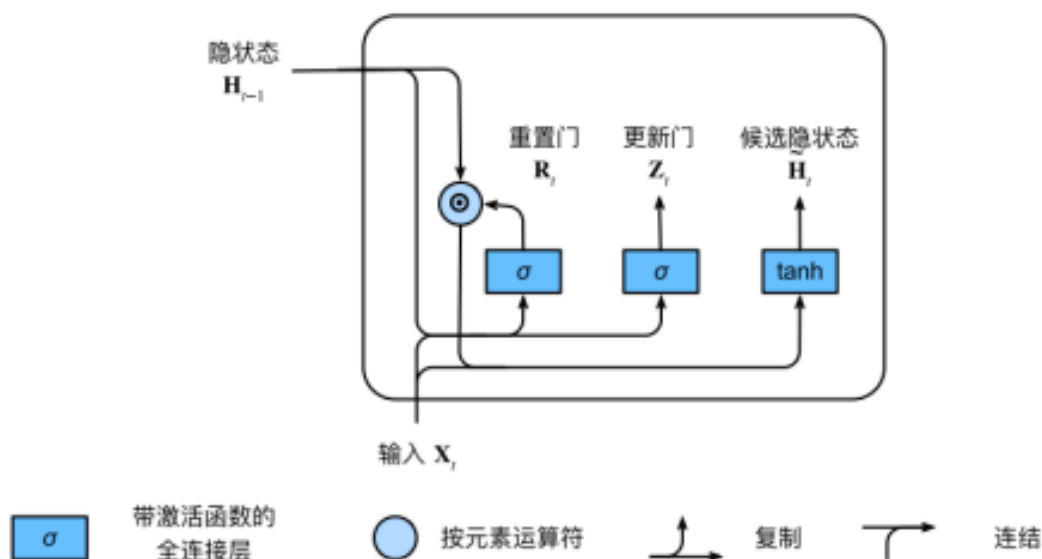
Step2 候选隐状态 $\tilde{H}_t$

候选隐状态可以理解为：如果我们决定更新隐状态，那么**新的内容大概应该是什么**？

计算公式为

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h)$$

其中符号 $\odot$ 为按元素乘积（可以粗糙理解为相乘）



看公式或者图能看出，**候选隐状态其实是由两个量综合而来的**：一个是当前输入 X_t （a 项），一个是原隐状态 H_{t-1} 和重置门 R_t 的乘积（b 项）。

从这里我们可以得出重置门的意义：

如果重置门 R_t 很接近 0，那么相当于 b 项几乎消失，候选隐状态几乎由 a 项也就是**新输入**决定；反过来，如果重置门 R_t 很接近 1，那么 b 项相当于就是原隐状态 H_{t-1} ，这时候选隐状态就类似于普通 RNN 的新状态的生成，会充分**利用过去的状态**。

所以，重置门 R_t ：控制过去的状态在候选隐状态中保留多少（忘不忘过去）

重置门数值越大，保留的原状态的成分越多

Step3 从候选隐状态 $\tilde{H}_t$ 到隐状态 H_t

隐状态 H_t 是真正得到的隐状态，其计算公式如下

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t$$

也即

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h)$$

其实很好理解，隐状态是候选隐状态 $\tilde{H}_t$ 和原隐状态 H_{t-1} 按照一定比例混合得到的。这个比例由更新门 Z_t 决定。

从这里我们可以得出更新门的意义：

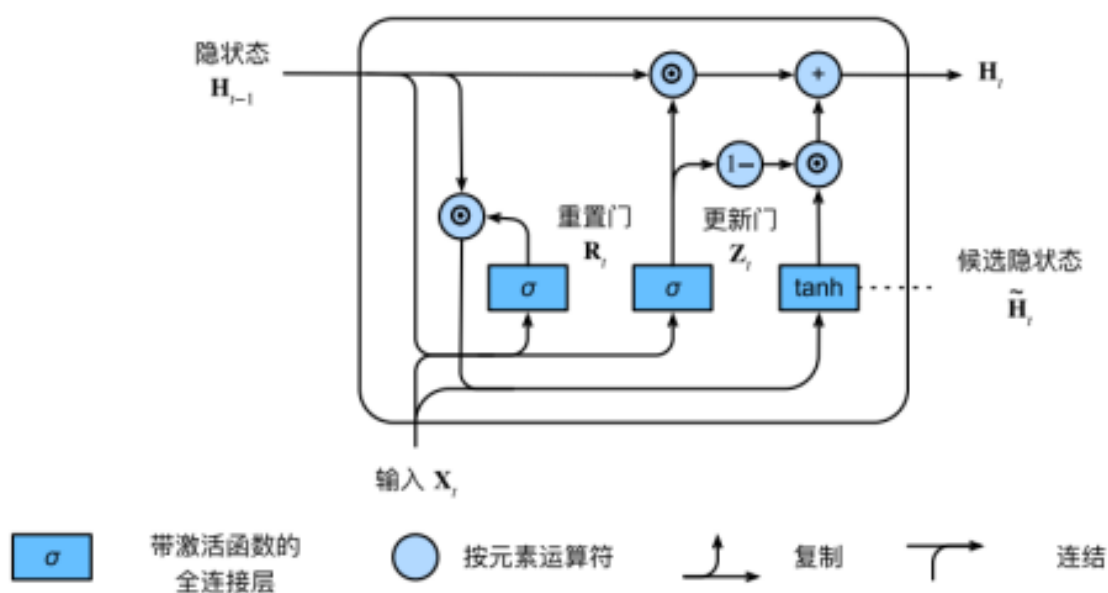
如果更新门接近 0，那么隐状态几乎就是候选隐状态

如果更新门接近 1，那么隐状态几乎就是原隐状态

所以，更新门 Z_t ：控制最终隐状态中旧状态，新状态各占多少（改不改现在的隐状态）

更新门数值越大，保留原状态越多

综合以上的三个 step，总体数据流动/计算见下图所示



总体理解：更新门是更加宏观的调控：调控新隐状态的两个成分的占比；重置门是更加精细的调控：调控其中一个状态具体保留了多少原状态。

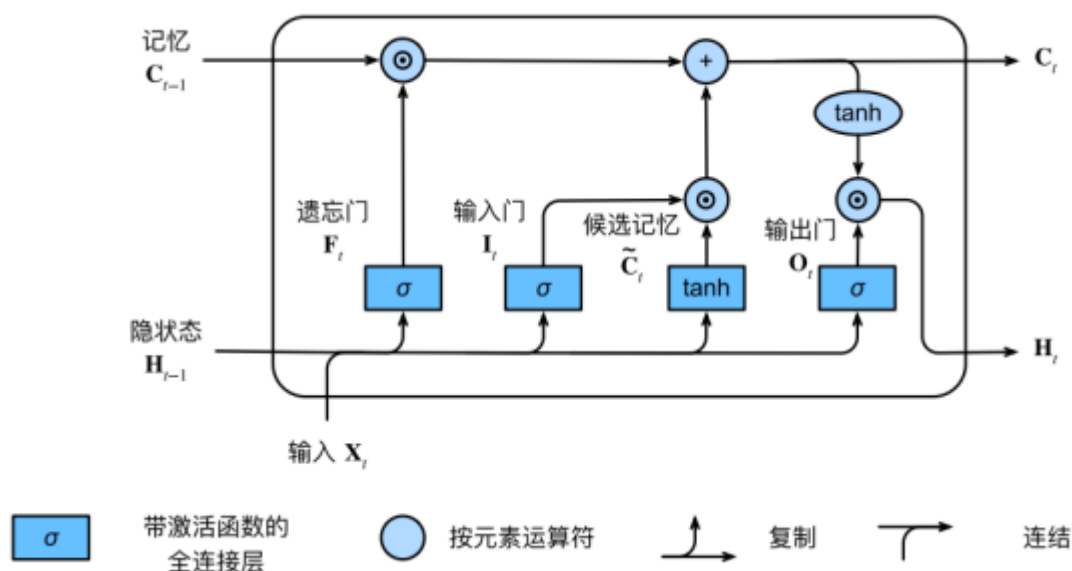
小结

- 门控循环神经网络可以更好地捕获时间步距离很长的序列上的依赖关系。
- 重置门有助于捕获序列中的短期依赖关系。
- 更新门有助于捕获序列中的长期依赖关系。

- 重置门打开（接近 1）时，门控循环单元包含基本循环神经网络；更新门打开（接近 1）时，门控循环单元可以跳过子序列。

长短期记忆网络（LSTM）

其实和 GRU 的目的是一样的：**都是想要解决传统 RNN 长期记忆难以保存的问题**，只不过其比 GRU 要稍微复杂一些。见下图：



梳理一遍逻辑：LSTM 除了隐状态以外，**还多了一个记忆**，每次输入原来的隐状态和记忆，更新输出新的隐状态和记忆。

LSTM 有三个门：**遗忘门 F_t** ，**输入门 I_t** ，**输出门 O_t** 。

一开始，根据隐状态和输入生成四个中间结果：三个门和一个候选记忆 $\tilde{C}_t$ 。计算式如下：

$$I_t = \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i)$$

$$F_t = \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f)$$

$$O_t = \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o)$$

$$\tilde{C}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

注意三个门用的激活函数是 sigmoid，取值为 $(0, 1)$ ，而候选记忆用的是 $\tanh$ ，取值为 $(-1, 1)$

接着，根据这些中间结果产生两个输出：新的记忆 C_t 和新的隐状态 H_t ，公式如下：

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

$$H_t = O_t \odot \tanh(C_t)$$

从公式看三个门的作用：

遗忘门 F_t ：控制新的记忆 C_t 里面有多少来自于旧的记忆 C_{t-1} （旧记忆会保留多少）

输入门 I_t ：控制新的记忆里面有多少来自于候选记忆 $\tilde{C}_t$ （当前产生的新记忆会被写入多少）

输出门 O_t ：控制新的隐状态（也就是对外输出）展示多少当前记忆 C_t 的内容（对外展示多少）

LSTM 和 GRU 的核心区别：GRU 的记忆和输出包含在同一个隐状态里，而 LSTM 将这两者分开了：单独的记忆元 C_t 和对外输出的隐状态 H_t

注意：LSTM 和 GRU 层的输出都是隐状态 H_t ，但是**整个模型的输出**还要在这个隐状态基础上经过一个全连接层得到：最终输出

$$Y_t = H_t W_{hq} + b_q$$

直观理解：LSTM 像是一个有“长期笔记本”的人：

- C_t ：长期笔记本，记录内部重要信息，但不对外输出。
- H_t ：当前说出口的内容。
- 输入门 I_t ：决定新信息要不要写进笔记本。
- 遗忘门：决定笔记本旧内容要不要擦掉。
- 输出门：决定笔记本里的内容这一步要不要说出来。

而 GRU 像是一个更简洁的记忆系统：

- 只有一个当前状态 H_t 。
- 更新门决定旧状态保留多少，新状态写入多少。
- 重置门决定生成新状态时，要不要参考旧状态（要参考多少旧状态）。

它没有单独的“笔记本”和“对外表达”之分。

LSTM 和 GRU 都能**缓解**传统 RNN 的梯度消失，因为它们通过门控机制建立了“旧状态直接保留”的路径，使**长期信息和梯度可以更稳定地跨时间步传播**；但它们不能彻底消除梯度爆炸，实际训练中仍常常需要梯度裁剪。

深度循环神经网络

上面的 LSTM 和 GRU 是在普通的 RNN 上，对于单层的处理结构上进行调整，而（最基本的）深度循环神经网络则是从另一个角度——**层数**上改进普通 RNN。结构如下图所示：

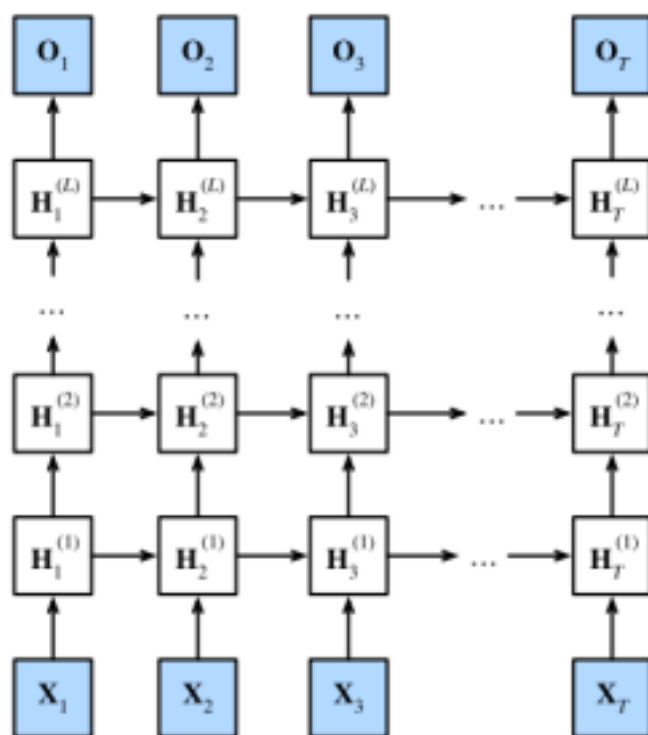


图9.3.1: 深度循环神经网络结构

改变：普通的 RNN 只有一层的隐状态层，而深度循环神经网络有多层（ L 层）

工作时，按照列进行处理：比如输入 x_2 ，则 x_2 和 $H_1^{(1)}$ 导出 $H_2^{(1)}$ ， $H_2^{(1)}$ 和 $H_1^{(2)}$ 导出 $H_2^{(2)}$ ，..... $H_2^{(L-1)}$ 和 $H_1^{(L)}$ 导出 $H_2^{(L)}$ ，然后 $H_2^{(L)}$ 再经过全连接层导出输出 O_2

公式如下：

$$\mathbf{H}_t^{(l)} = \phi_l(\mathbf{H}_t^{(l-1)} \mathbf{W}_{xh}^{(l)} + \mathbf{H}_{t-1}^{(l)} \mathbf{W}_{hh}^{(l)} + \mathbf{b}_h^{(l)})$$

$$\mathbf{O}_t = \mathbf{H}_t^{(L)} \mathbf{W}_{hq} + \mathbf{b}_q$$

注意：隐状态层之间的参数是不一样的（向上，右的箭头，不同行的箭头的参数不同）；同一隐状态层随时间推移的参数保持不变（向上，右的箭头，同一行的参数都相同）

也就是说，参数量从传统 RNN 的 5 个（ $L = 1$ ）拓展到了 $3L + 2$ 个

优点：就像加深了的多层感知机一样，**表达能力更强**，可以学习更复杂的关系，可以学习不同层次的序列特征。

注意：上述深度循环神经网络只是最基础的，从传统 RNN 上增加层数得到的

实际上，还可以进一步改造为 LSTM 或者 GRU

回看 LSTM 和 GRU，其实就是复杂化了输入和隐状态如何导出新的隐状态（LSTM 相当于是每一层都有记忆和隐状态两条线并行，但是只有隐状态在层和层之间传播“织成网”，记忆在同一层内

部沿时间传播"一根根线")，两个体系是兼容的

双向循环神经网络

为什么需要这个？在前面，我们的模型主要解决的是，已知一段序列，预测其后面的内容。而现实中还有其他任务，例如**完形填空**：需要结合前后文来确定空里填什么。此时就需要双向循环神经网络，从前后两个方向进行推导

具体结构如下图所示

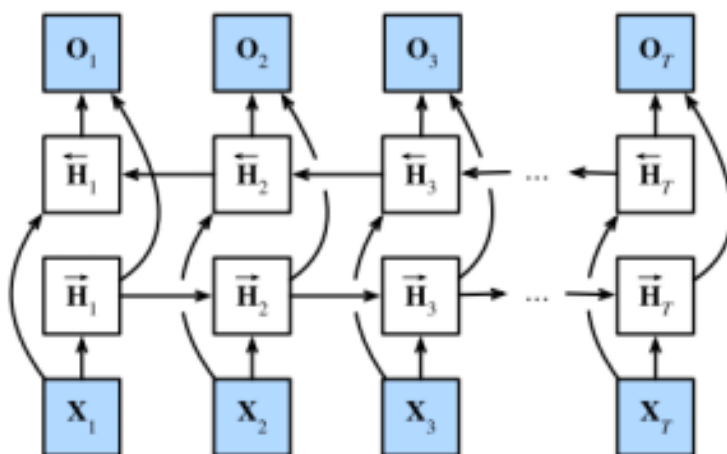


图9.4.2: 双向循环神经网络架构

其实可以理解成两个一层的传统 RNN 反向叠加：正向（从左到右）输入和之前正向隐状态导出新的正向隐状态；反向（从右到左）输入和之后反向隐状态导出新的反向隐状态。

然后同一下标不同方向的两个隐状态进行拼接，得到此处的隐状态，然后经过全连接层得到输出。

具体公式：（上标右箭头的是正向，左箭头的是反向）

$$\vec{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(f)} + \vec{H}_{t-1} \mathbf{W}_{hh}^{(f)} + \mathbf{b}_h^{(f)})$$

$$\overleftarrow{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh}^{(b)} + \overleftarrow{H}_{t+1} \mathbf{W}_{hh}^{(b)} + \mathbf{b}_h^{(b)})$$

$$H_j = [\vec{H}_j; \overleftarrow{H}_j]$$

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q$$

举例：以下标为 2 的为例：正向：输入 X_2 和正向隐状态 $\vec{H}_1$ 导出 $\vec{H}_2$ ；反向：输入 X_2 和反向隐状态 $\overleftarrow{H}_3$ 导出 $\overleftarrow{H}_2$ ； $\vec{H}_2$ 和 $\overleftarrow{H}_2$ 拼接成 H_2 ； H_2 经过全连接层导出输出

注意：不能将双向 RNN 运用在普通的语言预测任务中，因为会在训练中看到未来的词，而不是只根据过去预测未来。如果使用会导致结果严重错误

机器翻译与数据集

下面我们介绍一个新的任务：**机器翻译**（机翻）。

区别是什么？翻译，简单来说就是**把一个输入序列转化为一个输出序列**，基于神经网络的机器翻译强调端到端的学习，所以需要和之前不同的方法来处理数据集。

数据预处理：机器翻译的数据集是很多个“句子对”，比如从英语到法语，左边是英语句子（源语言），右边是对应的法语句子（目标语言）。首先要规范化（统一小写，空格），然后按照词来进行词元化，建立词表。然后，要添加一些特殊词元来规范化：

1. `<unk>`，表示未知词元。在词表中出现次数过少的单词可以变成 `<unk>` 来减少词表大小
2. `<eos>`，表示结束，接在原始句子的句尾
3. `<pad>`，表示填充空位，当上一步完成后还没有达到设定的长度，就在后面补上相应数量的 `<pad>`，但是**不计入有效长度**
4. **截断**：如果第二步完成后已经超出设定长度，就直接截断（可以句子不完整，可以没有 `<eos>`）

举一个例子：

原始文本：`I'm home. Je suis chez moi .`

1. 预处理为：`i'm home . je suis chez moi .`（转小写，统一空格）
2. 划分为源语言和目标语言：

```
source: i'm home .
```

```
target: je suis chez moi .
```

3. 词元化：（下一步转化为索引（即数字）略去，因为这一步需要根据整个词表来决定）

```
source = ["i'm", 'home', '.']
```

```
target = ['je', 'suis', 'chez', 'moi', '.']
```

4. 加 `<eos>` 结束符：

```
source = ["i'm", 'home', '.', <eos>]
```

```
target = ['je', 'suis', 'chez', 'moi', '.', <eos>]
```


5. 用 `<pad>` 填充至设定长度（这里设为 8）

```
source = ["i'm", 'home', '.', <eos>, <pad>, <pad>, <pad>, <pad>]
```

```
target = ['je', 'suis', 'chez', 'moi', '.', <eos>, <pad>, <pad>]
```

但是不改变有效长度： `source_valid_len = 4` , `target_valid_len = 6`

编码器-解码器架构

编码器-解码器架构是**"输入序列 → 编码表示 → 输出序列"** 的通用序列转换框架，**不是机器翻译专用**；机器翻译只是最典型、最容易理解的例子。在机器翻译的应用中，上一节处理好的数据集就从架构图中左边输入。

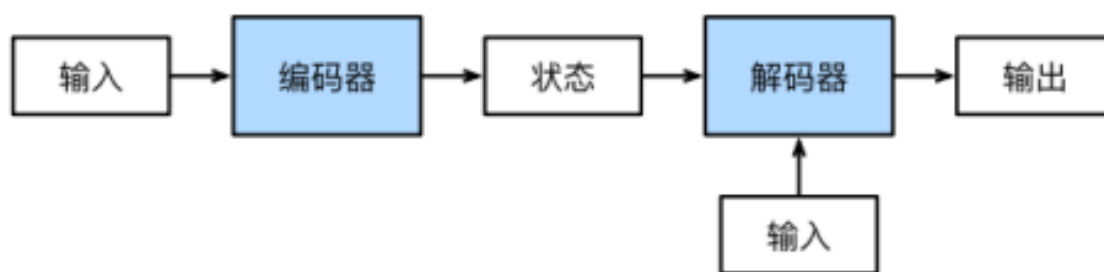


图9.6.1: 编码器-解码器架构

（注意，解码器除了接受编码器给的状态，自己也有输入）

序列到序列学习（seq2seq）

这一节，我们采用两个 RNN 的编码器和解码器，并将其运用到序列到序列学习中（sequence to sequence, seq2seq）

1. 编码器：就是将长度可变的输入序列 $\{x_t\}$ 转化为形状固定的**上下文变量** c （注意这里的长度可变指的是未经处理的原文本，真正输入编码器的是处理过的规范化 token 序列）

做法很简单，就是类似最简单的 RNN 的隐状态更新，每次用输入和旧隐状态生成新的隐状态，即

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$$

然后，编码器通过选定的函数 q ，将所有时间步的隐状态转换为上下文变量 c ：

$$\mathbf{c} = q(\mathbf{h}_1, \dots, \mathbf{h}_T)$$

本节中采用最简单的做法： $c = h_T$ （也就是只看最后一次的隐状态）

编码器在训练和预测中都是一样的，都要把源语言的句子送进去得到一个 c

2. 解码器：这个地方比较复杂。简单来讲，解码器做的事情和前面的预测下一个单词是差不多的，只不过输入以及对输入的特殊处理有些不同。

先看训练时：此时，解码器已经拿到编码器给他的 c ，现在需要接受输入来预测输出：

训练时给解码器的输入的前面要加上特殊词元 $\langle \text{bos} \rangle$ 表示开始；

同时，解码器自身也维护一个隐状态 s ，包含了 c 和已输入文本的信息。

注意：训练时给一个输入出一个输出，但是即使 $t - 1$ 的输出错了， t 时刻**仍然给正确的输入（强制教学，teacher forcing）**

举个例子：假设编码器已经根据源语言输入：They are watching . $\langle \text{eos} \rangle \langle \text{pad} \rangle \langle \text{pad} \rangle$ 生成了对应的 c 。目标语言的预期输出为 Ils regardent . $\langle \text{eos} \rangle \langle \text{pad} \rangle \langle \text{pad} \rangle$

第一步：给预期输出前面加上一个 $\langle \text{bos} \rangle$ ，然后 $\langle \text{bos} \rangle$ 作为输入 y_0 进入解码器。（也就是说，解码器手里现在有三个东西： c, s_0, y_0 ）（ s_0 就是 c ）接着，**解码器用 GRU 将这三者生成 s_1 ， s_1 经过全连接层得到预测的结果 $\hat{y}_1$**

第二步：加了 $\langle \text{bos} \rangle$ 的预期输出的第二个 token 作为输入 y_1 进入解码器。（也就是说，解码器手里现在还是三样： c, s_1, y_1 ）。然后重复上面的步骤，生成 s_2 并预测 $\hat{y}_2$

预测时： $\langle \text{bos} \rangle$ 作为 y_0 输入，预测 $\hat{y}_1$ 。注意此时和训练时的**主要区别**：**训练时不管 $\hat{y}_1$ 是否正确，都在下一步时输入正确的输入 y_1 ，而预测时只能将上一步预测的 $\hat{y}_1$ 作为下一步的输入**（训练时，有一个老师在你身边，当你偏离正确答案时把你扶正；测试时，没有老师，你只能相信自己前一步的结果一步步向后预测）

预测的结束：当预测到结束标志 $\langle \text{eos} \rangle$ 时结束预测

计算损失：在解码器给出预测时，输出的 s_t 在预测 $\hat{y}_t$ 的过程中，实际上和之前的“分类问题”很像： s_t 经过全连接层后归一化得到这个地方预测为某词元的概率表，然后 $\hat{y}_t$ 选取该表中概率最大的作为预测 token。**计算单次预测的损失，就是交叉熵损失**：模型预测正确的词元的概率的负对数（ $-\log P(y_{t,\text{correct}})$ ）。整句的损失就是所有单次预测损失的平均值

也就是说：模型越相信正确答案，损失越小；模型越不相信正确答案，损失越大。

目标语言预期输出的有效长度的意义：**计算损失以及反向传播时不计入 $\langle \text{pad} \rangle$ 的预测结果**（因为占位符本身就没有意义）也就是说，算损失只算到预测出 $\langle \text{eos} \rangle$ 为止，后面的不算。

还有一个指标 **BLEU**，用来评测最终生成的结果和标准答案差多远。具体公式很复杂这里略去，主要思想：如果预测句子和真实句子有很多词、很多连续词组都匹配，BLEU 就高；如果不匹配，

BLEU 就低。BLEU 还会**惩罚太短的翻译**。比如真实答案很长，但模型只输出一个词，即使这个词对了，BLEU 也不会很高。

束搜索

从上面的流程中很小一点入手："每次预测出概率表后，直接选取概率最大的作为预测词元"

这看起来很理所应当，也符合我们之前用这种思路做过的事情：图片识别

但是，区别在于，每次要识别的图片之间没有关联，当然可以无脑选择概率最大的

但是，**当选取了一个预测词元后，第二个词元的概率分布会因此而改变**

举例（每一列都是当前时间步选取对应词元的预测概率）：

时间步	1	2	3	4	时间步	1	2	3	4
A	0.5	0.1	0.2	0.0	A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.2	0.2	B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.4	0.2	C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.2	0.6	<eos>	0.1	0.2	0.1	0.6

情况一（每次都选取当前概率最大的）：组成 `ABC<eos>`，其总概率为 $0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$

情况二（在第二个时间步选择了第二大概率的 C，导致后续概率发生变化。此时再选概率最大的）：组成 `ACB<eos>`，其总概率为 $0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$ ，比前者大

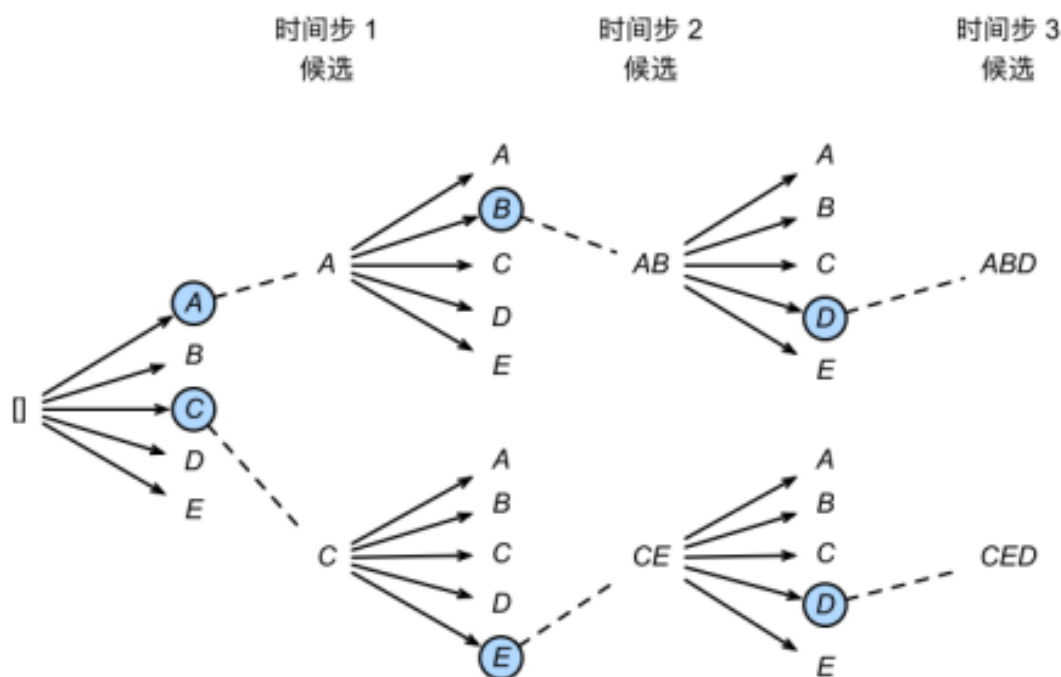
从上面的例子可以看到，**每次都选取最大概率的（称之为贪心算法），不一定能得到全局最优解。**

如何得到全局最优解呢？唯一可行的是**穷举搜索**：每一个时间步都记录所有可能的情况，最终能得到全局的所有情况，当然能得到最优解。但是有致命的问题：开销太大不值当。

所以，提出两种方案的折中方案：**束搜索**：

首先提出**束宽 (beam size) k** ：每一步，保留最好的 k 个结果。（与之对应的，**贪心搜索其实就是 $k = 1$ 的束搜索**）

举例：假设 $k = 2$ 。第一步预测时，保留 2 个当前最好的结果；第二步预测时，从这 2 个结果出发进行预测，**然后从这些里保留 2 个最好结果**。重复以上流程。如下图所示



什么是"好"? 用损失 loss 来刻画：表达式如下（loss 一定是正数，越小越好）

$$\text{Loss} = -\text{score} = -\frac{1}{L^\alpha} \sum_{t'=1}^L \log P(y_{t'} \mid y_1, \dots, y_{t'-1}, \mathbf{c})$$

其实就是：**每一步预测正确的概率的负对数之和**，然后前面乘上一个和序列长度相关的系数

其中 L 是序列长度， α 为可以人工调整的超参数，一般取 0.75。

为什么要有这一项？如果没有，**短序列天然有优势**。为了惩罚过短的序列，前面乘上这个系数，长序列会比短序列乘一个更小的数，来缩小 loss。

注意 1：后续时间步确定 k 个候选是按照**当前该序列的 loss**（越小越好）来确定而不是从第一步到现在所有概率乘积比大小/当前这一步的概率。

注意 2：**束搜索只在预测时执行**，而训练时还是只保留当前预测最大概率的 token（因为训练时有 teacher forcing 机制，不同时间步之间的预测结果没有关系，没必要用束搜索）

注意 3：束搜索的停止机制：当保留下来的路径以 `<eos>` 结尾，则该路径不再延伸，并将该路径纳入最终选择的路径之一；当所有的保留路径都以 `<eos>` 结尾，则停止。从所有这些保留下来的，以 `<eos>` 为结尾的路径里面选出最好的（loss 最低的）作为最终结果。

举例：（ $k = 2$ ）

第一步预测最好两个为 A, B,

第二步最好两个是 `A<eos>`，BC，则 `A<eos>` 保留为最终结果候选不再延伸，

第三步从 BC 继续预测最好为 BCD, BCE

(注意有的时候已经结束的候选也会占用名额, 也就是说这里可能只会保留一个预测结果)

第四步从 BCD, BCE 继续预测最好为 BCD<eos>, BCE<eos>

则此时结束, 从 A<eos>, BCD<eos>, BCE<eos> 中选取 loss 最小的为最终结果。

束搜索通过灵活选择束宽, **在正确率和计算代价之间进行权衡。**